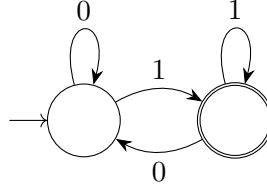
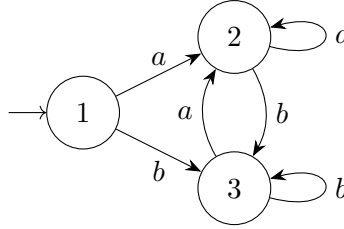


Computability 2024-2025 - Exam A Solution

1. (a) $L(A) = (0 + 1)^* \cdot 1$
 (b)



2. (a)



- (b) Each state in A' corresponds to a state in A and to a subset $S \subseteq [k]$ of colors visited so far.

$$Q' = Q \times 2^{[k]}$$

$$q'_0 = \langle q_0, \{f(q_0)\} \rangle$$

$$F' = Q \times \{[k]\}$$

For every $q \in Q$, $S \subseteq [k]$, and $\sigma \in \Sigma$, the transition respects δ , and the set of visited colors is updated accordingly:

$$\delta'(\langle q, S \rangle, \sigma) = \langle \delta(q, \sigma), S \cup \{f(\delta(q, \sigma))\} \rangle$$

3. $S \rightarrow 1S \mid S0 \mid 0S1 \mid \$$

4. For every $L_1 \in \text{RE}$ and $L_2 \in \text{coRE}$, we have $L_1 \setminus L_2 \in \text{RE}$: By definition, $L_2 \in \text{coRE}$ implies $\overline{L_2} \in \text{RE}$. Since $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$ and RE is closed under intersection, we have $L_1 \setminus L_2 \in \text{RE}$.
5. We show that $L \in \overline{\text{RE} \cup \text{coRE}}$ by a reduction from ALL_{TM} . Since the latter is not in RE and not in coRE, by the reduction theorem the same holds for L .

(I) Construction: Define $f(\langle M \rangle) = \langle M, T_{\text{all}} \rangle$, where T_{all} is a TM that immediately accepts every input, i.e. a TM with $q_0 = q_{\text{acc}}$.

(II) Correctness: Every word $w \in \Sigma^*$ can be written as $\text{rev}(\text{rev}(w))$, so we have $\text{rev}(\Sigma^*) = \Sigma^*$. Hence $\langle M \rangle \in \text{ALL}_{\text{TM}}$ iff $L(M) = \Sigma^*$ iff $L(M) = \text{rev}(L(T_{\text{all}}))$ iff $\langle M, T_{\text{all}} \rangle \in L$.

(III) Computability: A TM can output $\langle M, T_{\text{all}} \rangle$ given $\langle M \rangle$ by encoding the constant $\langle T_{\text{all}} \rangle$ next to $\langle M \rangle$. We have seen that this can be implemented with a TM.

6. We show that $L \in \text{RE} \setminus \text{R}$. To see that $L \notin \text{R}$, we can use Rice's theorem. The existence of $n \geq 0$ such that $\Sigma^n \subseteq L(M)$ depends only on the language of M , so it is a semantic property. The property is also non-trivial: a TM T_{all} that accepts every input satisfies it, whereas a TM $T_{\emptyset}$ that rejects every input does not.

To see that $L \in \text{RE}$, we define a TM T that recognizes L by running as follows on input $\langle M \rangle$:

- For $i = 0, 1, 2, \dots$:
 - For $n = 0, 1, 2, \dots, i$:
 - Check whether every word of length n is accepted by M within i steps. That is, for every $w \in \Sigma^n$, run M on w for at most i steps. If M does not accept, continue to the next n .
 - If M accepted all words of length n , accept.

We show that $L = L(T)$. In one direction, suppose $\langle M \rangle$ is accepted by T . By T 's behavior, this only happens if some $n \geq 0$ was found such that M accepts every word of length n . Hence $\langle M \rangle \in L$.

In the other direction, suppose $\langle M \rangle \in L$. Let $n' \geq 0$ be such that M accepts every word of length n' . Let t be the maximal number of steps M takes to accept a word of length n' . When T reaches the iteration $i = \max(n', t)$ and within it the iteration $n = n'$, it runs M on every word of length n' for t steps, and discovers that M indeed accepts all of them, so T accepts. Hence $\langle M \rangle \in L(T)$.

7. We show that L is NP-complete. To see that $L \in \text{NP}$, we define a polynomial witness for $\langle \varphi \rangle$ to be a satisfying assignment g such that at least half of the variables are assigned true. The polynomial verifier works as follows on a pair $\langle \varphi, g \rangle$: it checks that g indeed satisfies φ ; and it counts the number of variables assigned true by g , and checks that they are at least half of the variables. We have seen that each of these procedures can be done by a TM in polynomial time. For NP-hardness, we show a polynomial reduction from CNF-SAT. Since the latter is NP-hard, the same is true for L .

(I) Construction: Define $f(\langle \varphi \rangle) = \langle \varphi' \rangle$ where $\varphi' = \varphi \wedge (y_1 \vee \dots \vee y_n)$, and $y_1, \dots, y_n$ are new variables, and n is the number of variables in φ .

(II) Correctness: Suppose $\langle \varphi \rangle \in \text{CNF-SAT}$. Then there is a satisfying assignment to φ . Extend this assignment to φ' by setting all the new variables $y_1, \dots, y_n$ to true. This satisfies the new clause, so it satisfies φ' . There are $2n$ variables in φ' , and at least n are assigned true by this satisfying assignment, so $\langle \varphi' \rangle \in L$.

In the other direction, suppose $\langle \varphi' \rangle \in L$. Then there is an assignment that satisfies all clauses in φ' , and in particular satisfies φ . Hence $\langle \varphi \rangle \in \text{CNF-SAT}$.

(III) Complexity: $\langle \varphi' \rangle$ can be computed given $\langle \varphi \rangle$ by adding n variables and a clause that contains each of them. Since n is polynomial in the input size, this can be implemented in polynomial time by a TM.

8. (a) The claim implies that $\text{P} = \text{PSPACE}$. Since we have seen $\text{P} \subseteq \text{NP} \subseteq \text{PSPACE}$ and $\text{P} \subseteq \text{coNP} \subseteq \text{PSPACE}$, it also implies $\text{P} = \text{NP}$, $\text{NP} = \text{coNP}$, and $\text{NP} = \text{PSPACE}$.

Suppose $\text{ALL}_{\text{NFA}} \leq_p \text{PATH}$. We have seen that ALL_{NFA} is PSPACE-hard, so for every $L \in \text{PSPACE}$, we have $L \leq_p \text{ALL}_{\text{NFA}} \leq_p \text{PATH}$. We have seen that $\text{PATH} \in \text{P}$, so by the reduction theorem and transitivity of reductions, $L \in \text{P}$ and hence $\text{PSPACE} \subseteq \text{P}$. The other direction holds regardless of the claim, so equality follows.

- (b) The claim implies that $\text{NP} = \text{PSPACE}$. Since PSPACE is a deterministic class and hence closed under complementation, it also implies that $\text{NP} = \text{coNP}$.

It is sufficient to show that $K \in \text{NP}$: if K is PSPACE-hard, then for every $L \in \text{PSPACE}$ we have $L \leq_p K$ and by the reduction theorem $L \in \text{NP}$, implying $\text{PSPACE} \subseteq \text{NP}$. The other direction holds regardless of the claim, so equality follows.

To show that $K \in \text{NP}$, we define a polynomial witness for $\langle G, s, k \rangle$ to be a clique of size k that contains s . The polynomial verifier works as follows: it checks whether the given vertices indeed form a clique of size k , and checks that s is in that clique. We have seen that such checks can be implemented by a TM in polynomial time.

- (c) The claim implies $L = NL$. It is sufficient to prove that K is NL-hard: then, assuming $K \in L$, for every $K' \in NL$ we have $K' \leq_L K$ so $K' \in L$ by the reduction theorem. For hardness, it is enough to show a logspace reduction from PATH to K , since PATH is NL-hard.

A concatenation of two languages is non-empty iff both languages are non-empty. Given a graph G and vertices s and t , the reduction constructs a DFA A such that there is a path from the initial state to an accepting state iff there is a path from s to t in G . It returns $\langle A, A \rangle$ (we could also return $\langle A, B \rangle$ where B is any DFA with a non-empty language).

- (I) Construction: Define $f(\langle G, s, t \rangle) = \langle A, A \rangle$, where $A = \langle Q, \Sigma, \delta, q_0, F \rangle$ is a DFA defined as follows based on $G = \langle V, E \rangle$. The states and the letters are the vertices, thus $Q = \Sigma = V$. The initial state is $q_0 = s$, and t is the only accepting state, thus $F = \{t\}$. For every $u, v \in V$, we define the transition:

$$\delta(u, v) = \begin{cases} v & (u, v) \in E \\ u & (u, v) \notin E \end{cases}$$

- (II) Correctness: Suppose $\langle G, s, t \rangle \in \text{PATH}$. Then there is a path $s, u_1, \dots, u_m, t$ in G . This path is an accepting run of A on the word $u_1 u_2 \dots u_m t$, so $L(A)$ is non-empty and hence $\langle A, A \rangle \in K$.
In the other direction, suppose $\langle A, A \rangle \in K$. Then $L(A) \neq \emptyset$, so there exists an accepting run of A , which is a path from s to t . Hence $\langle G, s, t \rangle \in \text{PATH}$.
- (III) Complexity: A TM can encode A by iterating over each vertex and each edge in the graph. Every vertex is encoded into Q and Σ . For every two vertices $u, v \in V$, the reduction outputs a corresponding transition depending on whether $(u, v) \in E$. This can be implemented in logspace by keeping a (constant) number of pointers in order to keep track of the current vertices and edge being checked.